

The Sandbox Defender's Handbook: A Systematic Approach to Evasion Countermeasures

Mohammed Siddiqui

Contents

1	Prerequisite Concepts: The Virtualization Vocabulary	5
2	Architecture & The Evasion Landscape	7
2.1	The Hardware Virtualization Layer	7
2.1.1	Type 1 vs. Type 2 Hypervisors	7
2.1.2	Side-Channel Susceptibility in Type 1 Architectures	8
2.2	The Five Information Flows	9
2.3	The Observer Effect	10
2.3.1	Inline Hook Detection: The 0xE9 Heuristic	10
2.3.2	Environmental Profiling: Dummy Processes and CPU Kinetics	10
3	Infrastructure & Lab Setup	11
4	Hardware & Instruction Spoofing	15
4.1	T01: CPUID Hypervisor Bit	15
4.1.1	Mechanism of the __cpuid(1) Check	15
4.1.2	Countermeasure: Hooking KVM_SET_CPUID2	15
4.2	T06: RDTSC Timing Analysis	17
4.2.1	The RDTSC Latency Check and Why LD_PRELOAD Fails	17
4.2.2	Mitigation Strategies	18
4.2.3	Stabilising the Virtual Machine	20
5	Virtual Device & Peripheral Masking	21
5.1	T08: Display and EDID Spoofing	21
5.1.1	The QueryDisplayConfig Check	21
5.1.2	Countermeasure: EDID Injection via libvirt XML	21
5.2	T07: Audio Hardware Detection	22
5.2.1	The waveOutGetNumDevs() Check	22
5.2.2	Countermeasure: Emulating ICH9 Audio	22
5.3	T02: MAC Address Spoofing	23
5.3.1	The GetAdaptersInfo OUI Check	23
5.3.2	Countermeasure: Hardware Vendor OUI Spoofing	23
5.4	T09: Network Isolation and Deceptive Routing	24
5.4.1	Network Environment Profiling	24
5.4.2	Countermeasure: Dual-VM Deceptive Gateway	24
6	Guest State Manipulation (Ring 3)	27
6.1	T03: Driver File Masking	27
6.1.1	The GetFileAttributes Check	27
6.1.2	Countermeasure: T03_countermeasure.ps1	28
6.2	T04 & T05: Process Inflation & Hiding	29
6.2.1	Process Snapshots and Counts	29

6.2.2	Countermeasure: T04_countermeasure.bat (Inflation)	29
6.2.3	Countermeasure: T05_countermeasure.bat (Hiding)	29
6.3	T10: System Aging & Artifact Generation	31
6.3.1	The Sterile Environment Heuristic	31
6.3.2	Countermeasure: T10_countermeasure.ps1 (System Aging)	31
7	Advanced Tooling & Live Patching	33
7.1	Deep Hypervisor Patching	33
7.1.1	Architectural Imperative: Modifying vmx.c	33
7.1.2	Manipulating CPU MSRs for True Stealth	33
8	Experimental Evaluation & Limitations	35
8.1	Testing Methodology	35
8.2	Pre/Post-Spoof Evaluation	35
8.3	Limitations	36
8.3.1	The Fragility of Ring 3 (User-Mode) Manipulation	36
8.3.2	Complexity of Hypervisor Instrumentation	37
8.3.3	Peripheral Masking Overhead	37
A	Raw Test Results and Evidence	39
A.1	T06: RDTSC Latency (Ring -1 Hypervisor Patching)	39
A.2	T08: Display and EDID Spoofing (Ring 0 Configuration)	40
A.3	T09: Network Isolation and Deceptive Routing	40

Chapter 1

Prerequisite Concepts: The Virtualization Vocabulary

For readers new to malware analysis and system engineering, it is crucial to understand the foundational terminology used throughout this handbook before diving into evasion techniques.

- **Virtual Machine (VM):** A software-based emulation of a physical computer. It acts as a secure, isolated container where an operating system can run independently of the host machine.
- **Sandbox:** A highly monitored, isolated environment (usually a VM) built specifically to execute and analyze suspicious files safely.
- **Hypervisor:** The core software that creates, runs, and manages Virtual Machines. It allocates physical resources (like CPU, RAM, and Disk Space) to the guest VMs.
- **CPU Rings (Privilege Levels):** Modern processors use a ring architecture to protect system stability.
 - **Ring 3 (User-land):** Where normal applications (like web browsers or malware) run. They have restricted access to hardware.
 - **Ring 0 (Kernel):** The operating system's core. It has full control over the system.
 - **Ring -1 (Hypervisor):** A special hardware-assisted level beneath the OS, allowing the hypervisor to manage multiple Ring 0 operating systems simultaneously.
- **VM-Exit:** When a guest VM attempts to perform a highly privileged hardware action, the CPU temporarily pauses the guest and hands control back to the Hypervisor (Ring -1) to process the request safely. Malware measures the time this takes to detect if it is being watched.

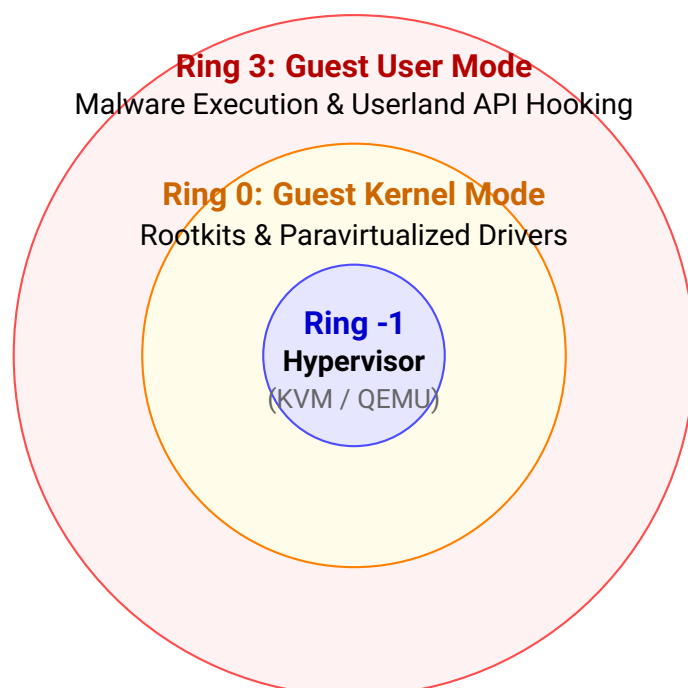
Chapter 2

Architecture & The Evasion Landscape

The effectiveness of a malware analysis lab is fundamentally bound by its proximity to physical reality. As hypervisors mature, the line between a virtualized environment and bare metal shrinks, yet a measurable difference remains. This chapter explores the architectural foundations of sandbox engineering, discussing the virtualization layer, the critical information flows targeted by both analysts and adversaries, and the *Observer Effect* that plagues dynamic analysis [1, 2].

2.1 The Hardware Virtualization Layer

Modern sandboxing architectures pivot on the selection of the hypervisor. This decision dictates the ring privileges of the analysis tools, the performance overhead and the environment's vulnerability to hypervisor detection techniques (VM exits).



2.1.1 Type 1 vs. Type 2 Hypervisors

Type 2 hypervisors (hosted), such as VMware Workstation or VirtualBox, run atop a host operating system. While they offer ease of deployment and extensive API support for taking snapshots and memory introspection, their reliance on a host OS introduces significant latency and

a broad attack surface for detection [3, 4]. The hardware abstractions are heavily emulated, making fingerprinting trivial for sophisticated malware via rudimentary checks (e.g. MAC addresses, specific registry keys, or driver artifacts).

Type 1 hypervisors (bare-metal), such as KVM (Kernel-based Virtual Machine) or Xen, execute directly on the hardware. By utilizing hardware-assisted virtualization extensions (Intel VT-x or AMD-V), Type 1 architectures execute guest instructions natively on the CPU. The hypervisor only intervenes during privileged operations, triggering a VM exit (VMX root operation). This architecture dramatically reduces the emulation footprint, rendering traditional artifact-based detection obsolete.

2.1.2 Side-Channel Susceptibility in Type 1 Architectures

Despite their stealth, Type 1 hypervisors are susceptible to hardware-level side-channel attacks. Because the guest OS and the hypervisor share physical silicon (caches, TLB, branch predictors), an adversary can infer the presence of the virtualization layer by measuring the difference between architectural states.

When a guest executes a privileged instruction (e.g. `CPUID`), control is trapped by the hypervisor. This context switch (from VMX non-root to VMX root and back) causes a measurable cycle penalty. Malware authors exploit this by executing an `RDTSC` (Read Time-Stamp Counter) instruction before and after a forced VM exit.

```
1 rdtsc           ; Read lower 32 bits into EAX
2 mov ebx, eax    ; Store start time
3 cpuid           ; Force a VM exit (if virtualized)
4 rdtsc           ; Read end time
5 sub eax, ebx    ; Calculate delta
6 cmp eax, 0xFFF  ; Compare against threshold
7 jg Detected_VM  ; If delta is massive, we are in a VM
```

RDTSC Timing Profiling

Furthermore, advanced evasion techniques leverage cache timing attacks (e.g. `FLUSH+RELOAD` on the L3 cache) to detect hypervisor activity or TLB profiling to identify memory pages locked by hypervisor introspection engines.

2.2 The Five Information Flows

To build an undetectable lab, you must control and monitor the five critical boundaries through which a process interacts with its environment. Malware targets these specific flows precisely because they represent the chokepoints of dynamic analysis.

1. **Dynamic Link Libraries (DLLs):** The traditional vector for user-mode hooking. Sandboxes inject monitoring DLLs into the guest process to intercept API calls at the IAT (Import Address Table) or EAT (Export Address Table) level. Malware circumvents this by manually mapping PE (Portable Executable) files into memory or dynamically resolving APIs [5, 6] via PEB (Process Environment Block) parsing.
2. **Syscalls (System Calls):** The boundary between Ring 3 (user mode) and Ring 0 (kernel mode). Because user-mode hooks are easily detected, modern malware utilizes *direct syscalls*. By bypassing `ntdll.dll` entirely [7] and moving the syscall number directly into the `EAX` register before executing the `syscall` instruction, malware evades user-mode sandboxes to its activity.
3. **Hardware:** The flow of opcodes executed by the CPU. Emulation-based sandboxes (e.g. QEMU-based platforms) attempt to monitor execution instruction-by-instruction. Malware disrupts this flow using polymorphic engines, opaque predicates, or anti-emulation loops that execute billions of meaningless instructions, effectively timing out the sandbox's processing window.
4. **Inter-Process Communication (IPC):** Modern operating systems rely heavily on ALPC (Advanced Local Procedure Call), RPC, and shared memory. Malware leverages IPC to perform process hollowing or atom-bombing. Sandboxes often fail to track execution context across IPC boundaries, allowing malware to offload malicious execution to legitimate, trusted processes (e.g. `explorer.exe`).
5. **Network:** The exfiltration and Command & Control (C2) boundary. Sandboxes employ TLS interception and simulated internet (e.g. INetSim). Malware targets this flow by pinning certificates, utilizing custom encryption over raw TCP/UDP, or employing DGA (Domain Generation Algorithms) that rely on temporal data the sandbox cannot accurately fake.

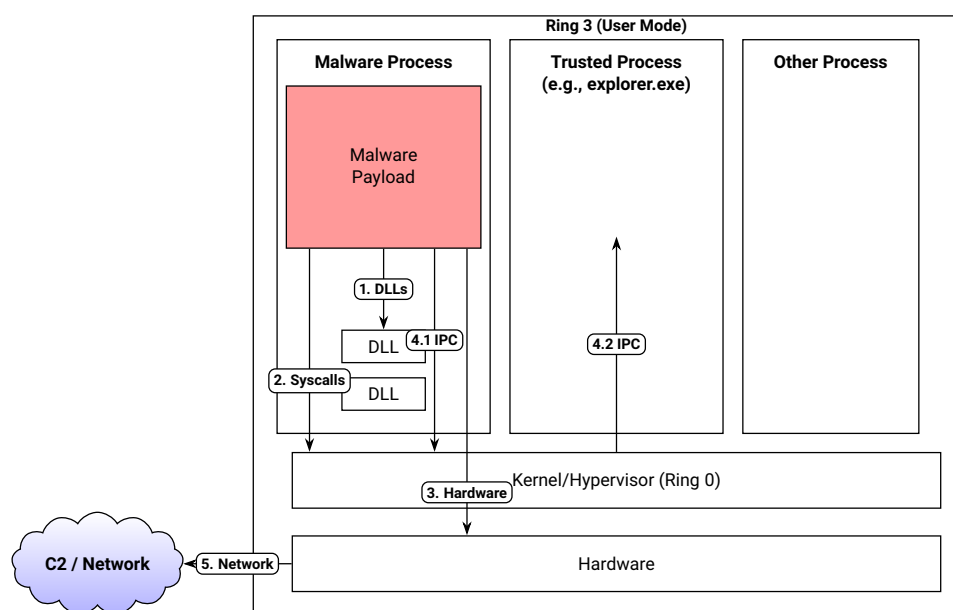


Figure 2.1: The Five Information Flows targeted by malware to evade dynamic analysis.

2.3 The Observer Effect

In quantum mechanics, the Observer Effect states that the mere act of observing a system alters its state. In malware analysis, the instrumentation required to monitor a sample inevitably leaves forensic artifacts. Sophisticated malware does not merely look for the sandbox; it looks for the *observer*.

2.3.1 Inline Hook Detection: The 0xE9 Heuristic

To intercept API calls without altering the IAT, sandboxes frequently employ inline hooking (detouring). The analysis engine overwrites the first five bytes of a target function (e.g. `VirtualAlloc`) with an unconditional relative jump (`JMP`, opcode `0xE9`) pointing to the sandbox's monitoring trampoline.

Malware counters this by reading the function's prologue directly from memory before executing it.

```
1 FARPROC pVirtualAlloc = GetProcAddress(GetModuleHandle("kernel32.dll"),
2                                     "VirtualAlloc");
3 unsigned char *pBytes = (unsigned char*)pVirtualAlloc;
4
5 // Check if the first byte is the JMP instruction (0xE9)
6 if (pBytes[0] == 0xE9) {
7     // Sandbox detected: Terminate or execute benign payload
8     ExitProcess(0);
9 }
```

Detecting Inline API Hooks

To build an undetectable lab, you must move away from Ring 3 inline hooks, relying instead on hardware breakpoints (DR registers), kernel-mode callbacks (e.g. `ObRegisterCallbacks`), or hypervisor-level memory introspection (VMI) using EPT (Extended Page Tables) manipulation.

2.3.2 Environmental Profiling: Dummy Processes and CPU Kinetics

Sandboxes automate human interaction by spawning dummy processes (e.g. moving the mouse, opening Word documents) to coax the malware into execution. Adversaries counter this by profiling the kinetic realism of the environment.

A real user generates organic, erratic CPU usage and process hierarchies. A sandbox executing a script to move the mouse generates predictable, mathematical CPU spikes. Malware actively monitors the CPU utilization using APIs like `GetSystemTimes` or by querying WMI. If the CPU usage remains unnaturally static—or if the ratio of foreground window changes to CPU cycles does not match human heuristics—the malware deduces the presence of artificial stimulation and aborts its execution chain.

Chapter 3

Infrastructure & Lab Setup

Building a resilient malware analysis sandbox requires an architectural approach that prioritizes stealth and deep visibility. The goal is to create an environment where we can observe advanced malware without triggering its environment-aware evasion techniques.

Recommended Architecture

For an advanced analysis lab, a Type 1 Hypervisor architecture (running directly on the hardware) is heavily preferred over Type 2 Hypervisors (like VirtualBox or VMWare Workstation). Type 1 hypervisors, such as KVM, are inherently stealthier because they require fewer OS-level drivers and introduce fewer user-land artifacts into the guest.

The recommended baseline consists of:

- **Host OS:** Ubuntu Linux.
- **Hypervisor:** QEMU/KVM managed via Libvirt.
- **Guest OS:** Windows 10/11 IoT LTSC or Windows Server Core with Desktop Experience.
(Note: Standard Windows Server Core lacks the necessary multimedia subsystems to support virtualized audio hardware requirements for evasion.)

Step 1: Initial Host Configuration (Ubuntu)

For beginners setting up their first sandbox, the following commands will install the necessary Type 1 hypervisor (KVM), the hardware emulator (QEMU), and a graphical interface (virt-manager) to easily deploy the Windows 11 guest.

Open your Ubuntu terminal and execute the following:

```
1 # Update system repositories
2 sudo apt update && sudo apt upgrade -y
3
4 # Install the core virtualization packages
5 sudo apt install qemu-kvm libvirt-daemon-system libvirt-clients bridge-
   utils virt-manager -y
6
7 # Add your user to the libvirt and kvm groups to avoid running
   everything as root
8 sudo usermod -aG libvirt $USER
9 sudo usermod -aG kvm $USER
```

Terminal: Install KVM & QEMU

Note: You must log out and log back in for the group changes to take effect.

Step 2: Deploying the Windows 11 Guest

Once the host is configured, you can download the official Windows 11 ISO from Microsoft [8]. Open the **Virtual Machine Manager** (`virt-manager`) application from your Ubuntu dash, click “Create a new virtual machine,” and follow the wizard.

Ensure you allocate at least **4 CPU cores** and **4096 MB of RAM** to ensure the Windows 11 installer passes its minimum hardware checks. Once the installation is complete, power off the VM. From here, we will begin modifying the underlying XML configurations to hide the environment. While KVM offers excellent stealth, it tightly integrates with the hardware, making it susceptible to Side-Channel timing attacks [9]. Defending against these checks requires advanced instrumentation at the hypervisor level.

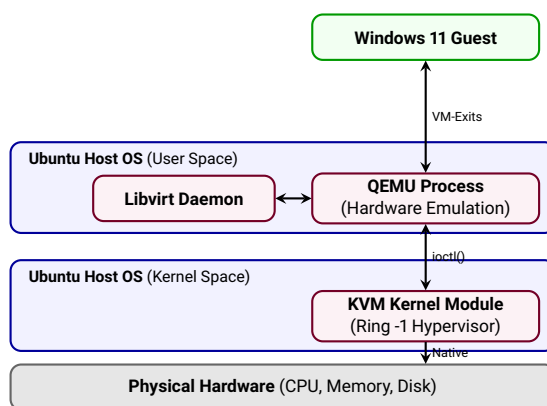


Figure 3.1: QEMU KVM Architecture

Hypervisor Instrumentation via Wrapper Scripts

To implement sandbox evasion countermeasures effectively, you must push your modifications as low in the architecture as possible. Rather than relying solely on noisy in-guest DLL injections, we can instrument the hypervisor process (QEMU) itself from the Linux host.

To achieve this, we use wrapper scripts that manipulate the execution environment of QEMU before it launches. A common technique is utilizing the `LD_PRELOAD` environment variable [10]. This forces the Linux dynamic linker to load a custom shared library (`.so`) before any others, allowing us to intercept and hook library functions used by the QEMU process.

Compiling the Hook Library

Before we can use `LD_PRELOAD`, we must compile our custom C code (`kvm_hook.c`) into a shared library object (`.so`). In your Ubuntu terminal, navigate to the folder containing your code and run the GNU C Compiler (`gcc`):

```

1 gcc -shared -fPIC -o kvm_hook.so kvm_hook.c
2 # Run QEMU with LD_PRELOAD
3 LD_PRELOAD=./kvm_hook.so qemu-system-x86_64 ...

```

- `-shared`: Instructs the compiler to produce a shared object rather than a standalone executable.
- `-fPIC`: Generates Position Independent Code, required for shared libraries.
- `-ldl`: Links the dynamic linker library, allowing us to use functions like `dlsym()` to intercept system calls.

Once compiled, you can inject it into the QEMU process using a wrapper script. Here is an example of a wrapper script (`run_QEmu.sh`): Here is an example of a wrapper script (`run_QEmu.sh`) used to inject a custom KVM hooking library into the QEMU process:

```
1 LD_PRELOAD=./kvm_hook.so qemu-system-x86_64 -enable-kvm -m 2048 \
2 -hda /media/firefly/VMs/TargetVM-W11-QEMU.qcow2
```

Listing 3.1: `source_code/T01/run_QEmu.sh`

By preloading `kvm_hook.so`, the analyst can dynamically modify hypervisor behavior such as intercepting `ioctl` calls to spoof hardware states or `CPUID` responses without having to recompile the entire QEMU/KVM source tree. This provides a flexible way to test countermeasures and bypass certain system-level execution restrictions dynamically.

Tracing Hardware Flows: The VM-Exits

Sophisticated malware will often bypass the OS entirely and execute raw assembly instructions (like `CPUID`) to query the CPU directly (Flow 3: The Instruction Flow). These instructions cause a “VM-Exit”—a context switch where execution is handed back from the guest VM to the host hypervisor to handle the request.

Standard utilities like `strace` cannot monitor these hardware-level VM-Exits. To trace these interactions from the Linux host, we rely on specialized profiling tools.

Capturing VM-Exits

We can use the `perf kvm` toolset to capture the exact frequency and types of VM-Exits generated by the Windows guest.

The following command demonstrates how to record all KVM events globally across the host:

```
1 perf kvm record -a -o vmexit_trace.data
```

Listing 3.2: `source_code/vmexit_capture.sh`

Note: Allow this to run while detonating the malware sample within the Windows 11 guest, then terminate the capture.

Analyzing the Trace Data

Once the data is captured, we need to read the generated binary trace file to identify the specific evasion checks the malware performed.

The following command generates a readable report from the trace data:

```
1 perf kvm report -i vmexit_trace.data
```

Listing 3.3: `source_code/vmexit_read.sh`

This report will expose specific exit reasons, such as `EXIT_REASON_CPUID` (often used to check the hypervisor bit) or `EXIT_REASON_HLT`. If malware forces a VM-Exit and measures the time it took using the `RDTSC` (Read Time-Stamp Counter) instruction, it can detect the sandbox because VM-Exits take thousands of cycles longer than bare-metal execution. Identifying these exits in your trace is the first step to mitigating side-channel timing attacks.

Chronological Logging with `ftrace`

For an exact chronological log of hardware interactions rather than a statistical summary, you can utilize `ftrace` or `trace-cmd`. By enabling the `kvm:kvm_exit` kernel tracepoints located in `/sys/kernel/debug/tracing/`, you can monitor the precise sequence of the malware's hardware queries.

Chapter 4

Hardware & Instruction Spoofing

This chapter details the countermeasures for CPU-level sandbox evasion techniques. Evasion techniques operating at the hardware or hypervisor level (Ring -1) often rely on querying the CPU directly to determine if the execution environment is a virtual machine. We will explore two primary methods used by malware: querying the CPUID hypervisor bit and analysing RDTSC instruction latency.

4.1 T01: CPUID Hypervisor Bit

4.1.1 Mechanism of the `__cpuid(1)` Check

The CPUID instruction allows software to discover details about the processor [11]. When executed with `EAX = 1`, the processor returns feature information in the ECX register. By convention, bit 31 of ECX is reserved as the "hypervisor present" bit. If this bit is set (1), it indicates that the code is running inside a virtual machine. Malware frequently uses the intrinsic `__cpuid(1)` to check this bit and halt execution if virtualization is detected [12].

4.1.2 Countermeasure: Hooking `KVM_SET_CPUID2`

To defeat this, we can intercept the communication between the hypervisor (QEMU) and the kernel module (KVM) before the VM even boots. QEMU uses the `ioctl` system call with the `KVM_SET_CPUID2` request to configure the virtual CPU's features [13]. By using `LD_PRELOAD`, we can hook this `ioctl` call and forcefully clear bit 31.

Here is the implementation using the provided `kvm_hook.c`

```

1 #define _GNU_SOURCE
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <dlfcn.h>
5 #include <stdarg.h>
6 #include <sys/ioctl.h>
7 #include <linux/kvm.h>
8
9 typedef int (*ioctl_func_t)(int fd, unsigned long request, ...);
10
11 int ioctl(int fd, unsigned long request, ...) {
12     static ioctl_func_t real_ioctl = NULL;
13     if (!real_ioctl) {
14         real_ioctl = (ioctl_func_t)dlsym(RTLD_NEXT, "ioctl");
15     }
16
17     va_list args;
18     va_start(args, request);
19     void *argp = va_arg(args, void *);
20     va_end(args);
21
22     if (request == KVM_SET_CPUID2) {
23         struct kvm_cpuid2 *cpuid = (struct kvm_cpuid2 *)argp;
24         for (int i = 0; i < cpuid->nent; i++) {
25             if (cpuid->entries[i].function == 1) {
26                 // Clear the Hypervisor Bit (Bit 31 of ECX)
27                 cpuid->entries[i].ecx &= ~(1 << 31);
28             }
29         }
30     }
31
32     return real_ioctl(fd, request, argp);
33 }

```

Listing 4.1: kvm_hook.c - Intercepting and patching CPUID

How it works: The dynamically loaded library intercepts all `ioctl` calls. When it detects QEMU initializing the virtual CPU's CPUID features (`KVM_SET_CPUID2`), it parses the `kvm_cpuid2` structure, finds the entry for `function == 1`, and applies a bitwise AND with the inverse of `1 << 31` to clear the hypervisor bit.

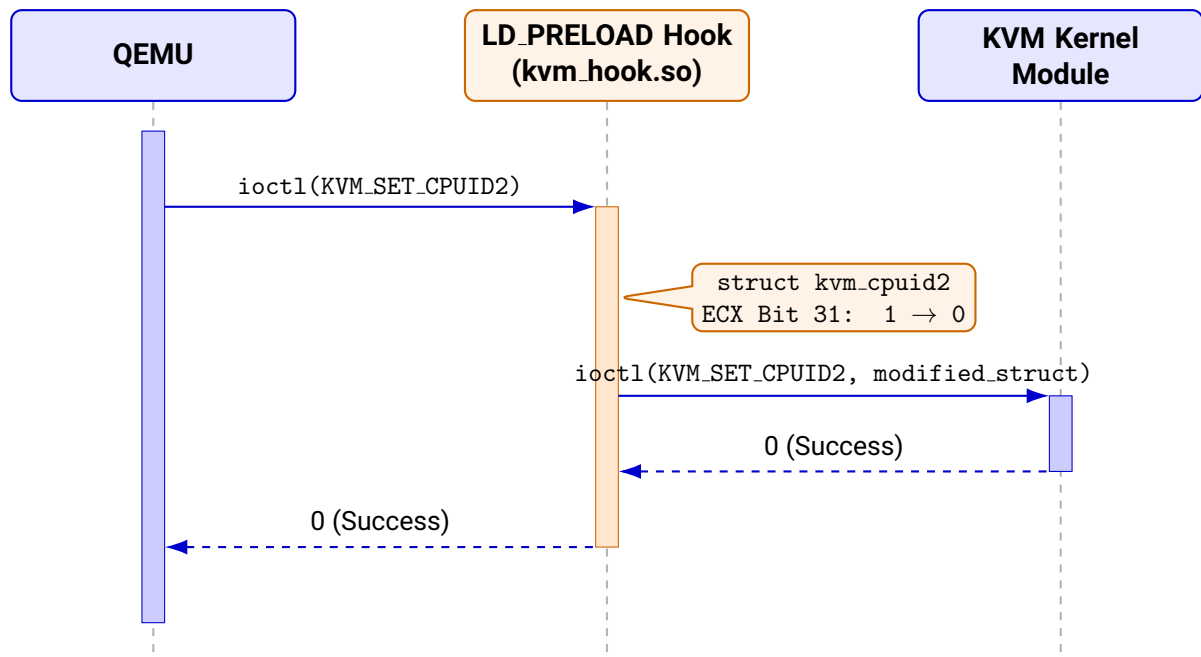


Figure 4.1: Intercepting the KVM_SET_CPUID2 IOCTL call during QEMU boot.

To compile and use this hook, utilize the provided `compile_so.sh` script:

```

1 gcc -shared -fPIC -o kvm_hook.so kvm_hook.c
2 # Run QEMU with LD_PRELOAD
3 LD_PRELOAD=./kvm_hook.so qemu-system-x86_64 ...

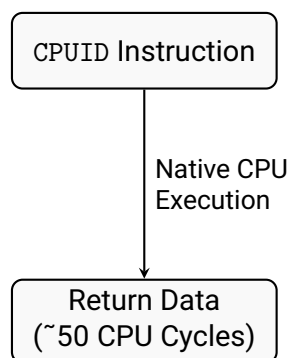
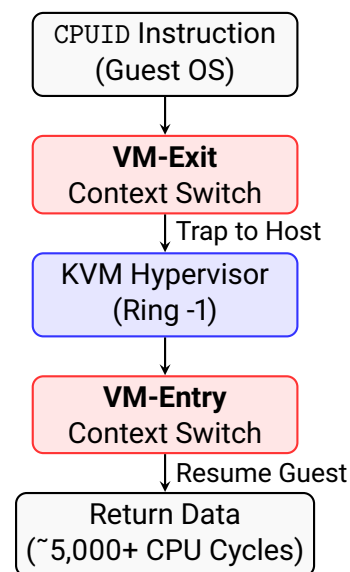
```

4.2 T06: RDTSC Timing Analysis

4.2.1 The RDTSC Latency Check and Why LD_PRELOAD Fails

The RDTSC (Read Time-Stamp Counter) instruction returns the number of CPU cycles since reset [14]. Malware measures the latency of executing instructions (often just back-to-back RDTSC calls) to detect virtualization. On bare metal, RDTSC takes a few dozen cycles. Inside a VM, if the hypervisor intercepts the instruction, it triggers a "VM-Exit," which can take thousands of cycles.

As noted in the engineering logs, attempting to bypass this using LD_PRELOAD will not work. LD_PRELOAD operates by intercepting library function calls, such as `printf` or `malloc` [10]. The `__rdtsc()` intrinsic acts as an inline assembly instruction. Because it is embedded directly as raw machine code in the `.text` section of the executable, there is no external library call to hook. The code bypasses the dynamic linker entirely.

Bare Metal Execution**Virtualized Execution****4.2.2 Mitigation Strategies**

There are two primary ways to neutralize this check:

1. Binary Patching: You can manually edit the compiled binary to neutralize the check. This involves finding the `0F 31` opcodes (the machine code for `RDTSC`). Once identified, you can replace the detection logic (e.g. `if (avg > 750)`) with `if (0)`, or simply `NOP` out the detection print.

2. Hypervisor Spoofing (TSC Pass-through): If the program is running inside a VM like QEMU/KVM, you can configure the hypervisor to handle the `RDTSC` trap and return a "fake" incremental value. Standard KVM TSC Offsetting does not change the latency of the instruction, but only makes the time look consistent [15]. To truly fix the latency spike caused by the VM-Exit delay, the hypervisor requires "Pass-through" or "Fixed" TSC rates.

As demonstrated in the custom `vmx.c` patch, we can force a VM-Exit on `RDTSC` and handle it intelligently within the hypervisor:

```

1 static u64 spoof_tsc(struct kvm_vcpu *vcpu)
2 {
3     static u64 guest_fake_tsc[256] = {0};
4     static u64 host_prev_tsc[256] = {0};
5     int id = vcpu->vcpu_id;
6     u64 host_now = rdtsc();
7     u64 real_delta;
8
9     // Initialization and delta calculation logic...
10    real_delta = host_now - host_prev_tsc[id];
11
12    // 500,000 cycles is roughly 0.1ms.
13    if (real_delta < 500000) {
14        // PROFILING DETECTED: Completely ignore VM-Exit latency.
15        // Force the clock forward by an amount that looks like a
16        // native CPUID+RDTSC combo.
17        guest_fake_tsc[id] += host_now % 643;
18    } else {
19        // NORMAL OPERATION: Let the fake clock catch up to physical
20        // time
  
```

```

19     guest_fake_tsc[id] += real_delta;
20 }
21
22     host_prev_tsc[id] = host_now;
23     return guest_fake_tsc[id];
24 }
25
26 static int handle_rdtsc(struct kvm_vcpu *vcpu)
27 {
28     u64 tsc = spoof_tsc(vcpu);
29     kvm_rax_write(vcpu, tsc & -1u);           // Lower 32 bits into EAX
30     kvm_rdx_write(vcpu, (tsc >> 32) & -1u); // Upper 32 bits into EDX
31     return kvm_skip_emulated_instruction(vcpu);
32 }

```

Listing 4.2: vmx.c - Spoofing the TSC delta

How it works: First, the KVM configuration is patched to set `CPU_BASED_RDTSC_EXITING`, forcing the CPU to trap every RDTSC call to the hypervisor. When the trap occurs, `handle_rdtsc` is invoked. The `spoof_tsc` function calculates the real time elapsed since the last call. If the delta is suspiciously small (under 500,000 cycles), it assumes the malware is actively profiling execution time. Instead of returning the massive latency of the VM-Exit, it adds a small, arbitrary number (`host_now % 643`) to the fake counter, perfectly mimicking the latency of bare-metal execution. If the delta is large, it assumes normal OS background tasks and increments normally to prevent the virtual machine's clock from desyncing.

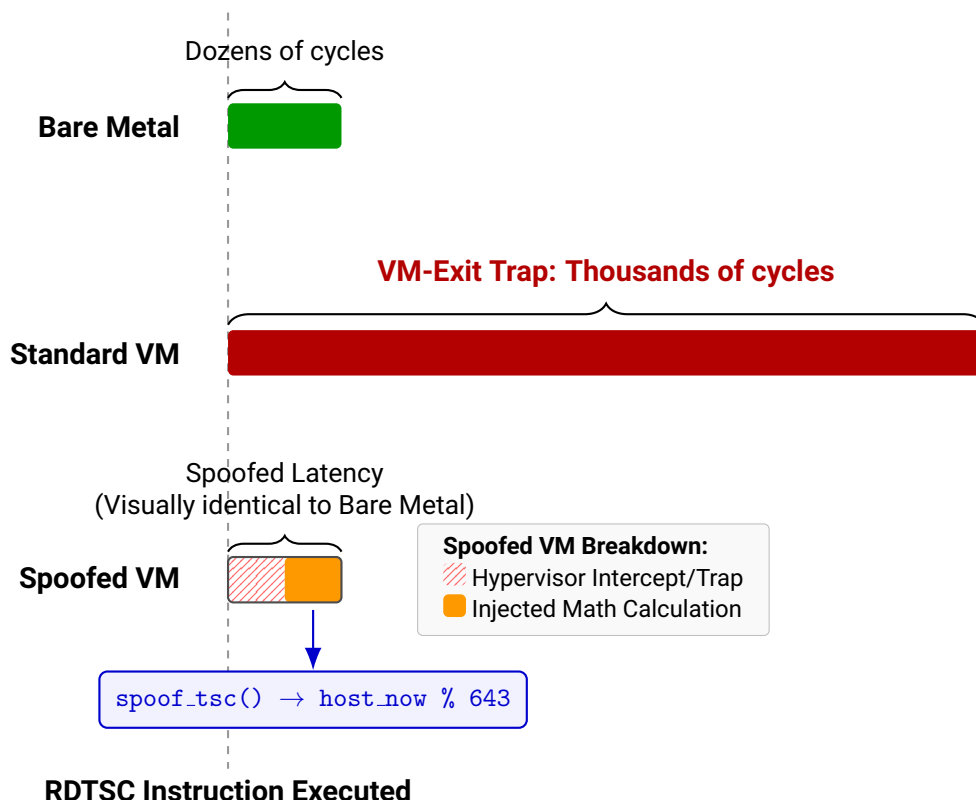


Figure 4.2: Comparative latency timeline: Bare Metal vs. Standard VM-Exit vs. Spoofed TSC.

4.2.3 Stabilising the Virtual Machine

Implementing Ring -1 hypervisor spoofing, such as the `RDTSC` patch, often introduces secondary instabilities. For example, Windows actively cross-references the standard Time Stamp Counter (TSC) against Hyper-V synthetic timers (`stimer` and `synic`). If the base TSC is warped by a custom kernel module but the synthetic timers are not, the guest OS detects the desynchronization. This causes severe regressions, including 100% host CPU spikes and sudden VM terminations.

To resolve this “Time Paradox” and stabilize the execution environment, you must explicitly disable the guest’s high-precision synthetic timing checks. This forces the Windows kernel to rely entirely on your spoofed TSC as its primary clock source.

Open your VM’s configuration using `virsh edit <vm_name>` and apply the following modifications to the `<features>` block and `<clock>` block:

```
1 <features>
2   <acpi/>
3   <apic/>
4   <hyperv mode='custom'>
5     <relaxed state='on'>/>
6     <vapic state='on'>/>
7     <spinlocks state='on' retries='8191'>/>
8     <vpindex state='on'>/>
9     <runtime state='off'>/>
10    <synic state='off'>/>
11    <stimer state='off'>/>
12  </hyperv>
13 </features>
14 <clock>
15   <timer name='hypervclock' present='no'>/>
16 </clock>
```

Listing 4.3: Libvirt XML - Disabling Hyper-V Synthetic Timers

Chapter 5

Virtual Device & Peripheral Masking

This chapter focuses on countermeasures for hardware discovery evasions. Advanced malware frequently interrogates the operating system for peripheral devices—such as displays, audio outputs, and network adapters—to determine if it is executing in a sterile, headless virtual machine. We will detail how to defeat these checks primarily through precise manipulation of the hypervisor’s `libvirt` XML configuration [16].

5.1 T08: Display and EDID Spoofing

5.1.1 The `QueryDisplayConfig` Check

In a standard Virtual Machine, the virtualized graphics adapter often reports generic monitor IDs (like “Generic Non-PnP Monitor”) and lacks valid Extended Display Identification Data (EDID). Malware uses Windows APIs like `QueryDisplayConfig` to interrogate the connected displays. If it finds unusual refresh rates, abnormally small active sizes, or missing EDID blocks [17], it assumes a sandbox environment and terminates [18]. While it is possible to use API hooking to intercept `QueryDisplayConfig` and force it to return a standard 1920x1080 resolution at 60Hz, this approach leaves artifacts in memory. A much stealthier, Ring-0 approach is to inject a raw, physical monitor EDID directly into the virtualized video adapter via the hypervisor.

5.1.2 Countermeasure: EDID Injection via `libvirt` XML

By extracting the EDID binary from a physical monitor (e.g. `real_monitor_edid.bin`), we can instruct QEMU to serve this exact data structure to the guest OS. The guest will natively parse it and believe a legitimate physical monitor is attached. To implement this, locate the `<video>` block in your `libvirt` XML and modify it to include the `<edid>` tag pointing to your binary payload:

```
1 <video>
2   <model type='qxl' ram='65536' vram='65536' vgamem='16384' heads='
3     1' primary='yes' />
4   <edid file='/opt/real_monitor_edid.bin' />
  </video>
```

Listing 5.1: Injecting physical EDID into QXL Video Adapter

5.2 T07: Audio Hardware Detection

5.2.1 The `waveOutGetNumDevs()` Check

Enterprise servers and automated malware sandboxes typically do not have physical audio output devices. Malware leverages this by calling multimedia APIs such as `waveOutGetNumDevs()` or utilizing the `IMMDeviceEnumerator` interface [19] to check for the presence of audio endpoints. If zero devices are returned, the malware knows no human user is listening and aborts execution.

5.2.2 Countermeasure: Emulating ICH9 Audio

To bypass this, we must ensure the hypervisor presents a virtual sound card to the guest OS, which Windows will automatically install drivers for. We can achieve this by adding a virtual Intel ICH9 High Definition Audio controller to our libvirt XML. Add the following configuration within the `<devices>` section of your XML:

```
1 <sound model='ich9'>
2   <address type='pci' domain='0x0000' bus='0x00' slot='0x1b'
      function='0x0' />
3 </sound>
4 <audio id='1' type='spice' />
```

Listing 5.2: Adding a virtual audio controller

This configuration binds the virtual sound card to the SPICE display protocol, satisfying the OS's requirement for a connected audio sink.

5.3 T02: MAC Address Spoofing

5.3.1 The GetAdaptersInfo OUI Check

Every network interface card (NIC) has a MAC address where the first three bytes signify the Organizationally Unique Identifier (OUI). Default virtual machines generate MAC addresses with OUIs registered to hypervisor vendors [20, 21]. For example, QEMU uses 52:54:00, VMware uses 00:50:56, and VirtualBox uses 08:00:27. Malware calls APIs like `GetAdaptersInfo` or `GetAdaptersAddresses` [19] to fetch the MAC address and compares the OUI against a hard-coded blocklist of virtual environments.

5.3.2 Countermeasure: Hardware Vendor OUI Spoofing

To defeat this check, the VM's libvirt XML must be modified to assign a MAC address belonging to a legitimate physical hardware vendor, such as Intel. Locate your `<interface>` block and manually override the `<mac address='...'>` tag. In this example, we spoof an Intel OUI (00:1B:21):

```
1      <interface type='network'>
2          <mac address='00:1b:21:5a:33:91' />
3          <source network='default' />
4          <model type='e1000e' />
5          <address type='pci' domain='0x0000' bus='0x01' slot='0x00'
6              function='0x0' />
7      </interface>
```

Listing 5.3: Spoofing a physical Intel MAC Address

Architectural Requirement: Successfully emulating this audio hardware and bypassing multimedia API checks relies heavily on the guest operating system's capabilities. The default Windows Server Core installation does not include the necessary multimedia subsystems or drivers to utilize the ICH9 virtual sound card. Therefore, to defeat audio hardware detection (T07), the guest VM must be deployed using a Windows IoT LTSC ISO or a Windows Server installation specifically configured with the "Desktop Experience" feature enabled.

5.4 T09: Network Isolation and Deceptive Routing

5.4.1 Network Environment Profiling

Advanced malware frequently profiles its network environment by checking the default gateway, subnet masks, or attempting to reach out to known public IP addresses (e.g., IP geolocation services). If the malware detects a standard, unmodified NAT network commonly associated with QEMU/KVM defaults (such as the 192.168.122.0/24 subnet on `virbr0`), or if it discovers its external IP address belongs to a datacenter rather than a residential ISP, it will identify the sandbox environment and halt execution.

5.4.2 Countermeasure: Dual-VM Deceptive Gateway

To mitigate this, we move beyond simple isolation and implement a high-interaction network trap using a Dual-VM architecture. A REMnux Linux virtual machine is deployed alongside the Windows target VM on a custom isolated subnet to act as a deceptive gateway. To ensure consistent hypervisor management, resource allocation, and seamless integration with the `libvirt` virtual networks, this REMnux VM should also be hosted directly within QEMU utilizing KVM mode.

First, we define an `isolated_network` in `libvirt` to break the default virtualization footprint and enforce a 192.168.100.0/24 subnet:

```

1 <network>
2   <name>isolated_network</name>
3   <uuid>11ff6963-591f-401b-9765-6d9458fe1476</uuid>
4   <bridge name='virbr1' stp='on' delay='0' />
5   <mac address='52:54:00:c8:41:55' />
6   <domain name='isolated_network' />
7   <ip address='192.168.100.1' netmask='255.255.255.0'>
8     <dhcp>
9       <range start='192.168.100.128' end='192.168.100.254' />
10    </dhcp>
11  </ip>
12 </network>

```

Listing 5.4: Libvirt XML - Isolated Network Profile

Rather than allowing the Windows VM to route out to the real internet, the REMnux VM (statically assigned to 192.168.100.135) intercepts all outbound traffic. To capture and redirect the malware's DNS queries, we clear UDP Port 53 on the REMnux host and deploy a custom Python DNS trap. This script forces all domain resolutions to point back to the analysis machine:

```

1 import socket
2
3 # Set this to your REMnux IP
4 IP = '192.168.100.135'
5 PORT = 53
6
7 sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
8 sock.bind(('0.0.0.0', PORT))
9
10 print(f"DNS Trap active on port {PORT}. Redirecting all traffic to {IP}...")
11
12 while True:
13     data, addr = sock.recvfrom(512)

```



```

14     # Simple DNS response header: Transaction ID + Flags (Standard
15       Response, No Error)
16     packet = data[:2] + b"\x81\x80\x00\x01\x00\x01\x00\x00\x00\x00"
17     # Question Section
18     packet += data[12:]
19     # Answer Section: Name (pointer to offset 12) + Type (A) + Class (
20       IN) + TTL (60s) + Data Length (4)
21     packet += b"\xc0\x0c\x00\x01\x00\x01\x00\x00\x00\x3c\x00\x04"
22     # The fake IP address
23     packet += bytes(map(int, IP.split('.')))
24     sock.sendto(packet, addr)
25     print(f"Captured request from {addr[0]} - Sent redirection to {IP}")
26 )

```

Listing 5.5: dns_trap.py - DNS Redirection Script

Once the DNS trap redirects the malware's traffic back to the REMnux gateway, simulation services like INetSim are used to serve deceptive payloads. For example, if the malware attempts to geolocate its IP to check if it is running in a datacenter, INetSim intercepts the HTTP request and serves a fabricated JSON response indicating a legitimate residential ISP connection:

```

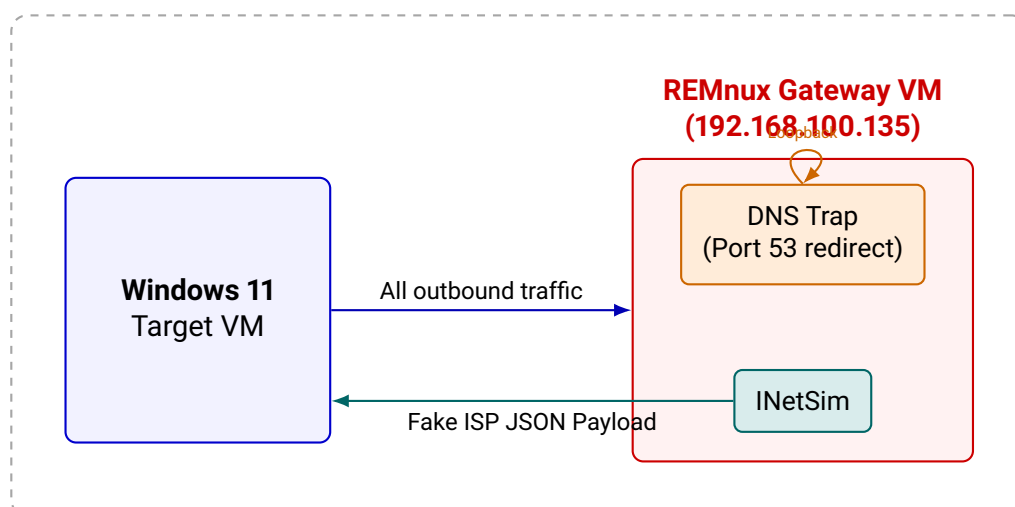
1 {
2   "ip": "73.142.12.5",
3   "org": "AS7922 Comcast Cable Communications, LLC",
4   "city": "San Francisco",
5   "region": "California",
6   "country": "US"
7 }

```

Listing 5.6: Fake ISP Payload (sample.html)

This architecture creates a localized "reality distortion field" at the network layer. The malware believes it is successfully communicating with the public internet from a residential host, completely unaware that all external flows are being captured, analyzed, and manipulated by the adjacent REMnux gateway.

Isolated Subnet (192.168.100.0/24)



Chapter 6

Guest State Manipulation (Ring 3)

This chapter focuses on user-land (Ring 3) state manipulation. Malware authors understand that automated sandboxes often present a “clean” but highly artificial environment. By probing the file system for virtualized drivers or counting active processes, malware can accurately fingerprint a sandbox without needing hypervisor or kernel-level access [22, 23, 24]. Here, we outline the countermeasures required to mask the guest operating system’s internal state.

6.1 T03: Driver File Masking

6.1.1 The `GetFileAttributes` Check

Virtual machines rely on paravirtualized drivers for performance—such as `viostar.sys` for `VirtualIO` storage or `vmmouse.sys` for VMware pointer integration. Malware frequently utilizes the `GetFileAttributes` Windows API [19] to probe the `C:\Windows\System32\drivers` directory [25]. If the API returns valid attributes instead of `INVALID_FILE_ATTRIBUTES`, the malware recognizes the virtualization driver’s footprint and halts execution to evade analysis. We cannot simply delete these drivers, as the guest operating system would fail to boot or lose network/storage access. Instead, we must rename the physical driver files and seamlessly update the Windows Registry so the Service Control Manager (SCM) loads them under a benign alias.

6.1.2 Countermeasure: T03_countermeasure.ps1

The following PowerShell script automates the masking of the VirtIO storage driver:

```

1 # Run as Administrator inside the Guest
2 $drivers = @("viostar", "vmmouse", "balloon")
3
4 foreach ($name in $drivers) {
5     $originalPath = "C:\Windows\System32\drivers\$name.sys"
6     $newName = "stornvme_ext_$name.sys" # Benign looking name
7     $newPath = "C:\Windows\System32\drivers\$newName"
8
9     if (Test-Path $originalPath) {
10        # 1. Rename the driver file
11        Move-Item -Path $originalPath -Destination $newPath -Force
12
13        # 2. Update the Registry ImagePath for the service
14        $regPath = "HKLM:\SYSTEM\CurrentControlSet\Services\$name"
15        if (Test-Path $regPath) {
16            Set-ItemProperty -Path $regPath -Name "ImagePath" -Value "
17                system32\drivers\$newName"
18            Write-Host "Successfully spoofed $name.sys to $newName"
19        }
20    }
21 }

```

Listing 6.1: T03_countermeasure.ps1 - Driver File Masking

Line-by-Line Logic:

- \$RegKey and \$DriverDir: Defines the target registry key for the VirtIO service and the physical driver directory.
- Rename-Item: Alters the .sys file on disk to a native-sounding name (e.g. storahci_v2.sys). This immediately thwarts any hardcoded GetFileAttributes path checks targeting the original driver name.
- Get-ItemProperty: Retrieves the ImagePath value, which Windows uses to locate the driver binary during the boot sequence.
- -replace and Set-ItemProperty: Performs a string replacement to swap the old file-name for the new one and commits it back to the Registry. The VM will now boot normally, completely hiding the VirtIO footprint from file-system scanners.

6.2 T04 & T05: Process Inflation & Hiding

6.2.1 Process Snapshots and Counts

Malware frequently calls APIs like `CreateToolhelp32Snapshot` or `EnumProcesses` [19] to iterate through the system's running processes. A standard bare-metal Windows installation typically runs over 100 to 150 background processes, whereas a freshly spun-up sandbox often runs fewer than 40. If the malware detects an unusually low process count, it assumes it is in an isolated, sterile environment and remains dormant [26]. Furthermore, if it enumerates the snapshot and discovers analysis tools like `wireshark.exe` or `x64dbg.exe`, it will immediately terminate.

6.2.2 Countermeasure: T04_countermeasure.bat (Inflation)

To defeat the process count check, we artificially inflate the system state by spawning hidden instances of standard Windows applications.

```
@echo off
:: Inflate the process list with benign applications
FOR /L %A IN (1,1,15) DO (
    start /MIN notepad.exe
    start /MIN calc.exe
)
echo [+] Process inflation complete.
```

Listing 6.2: T04_countermeasure.bat - Process Inflation

Line-by-Line Logic:

- `@echo off`: Suppresses command echoing to keep the execution silent and clean during the VM provisioning phase.
- `FOR /L %A IN (1,1,15) DO`: Initiates a standard loop that iterates 15 times.
- `start /MIN`: The `start` command launches the target executables (`notepad.exe` and `calc.exe`) asynchronously. The `/MIN` flag forces the windows to open minimized, ensuring they do not clutter the visible desktop while still registering as active processes in the kernel's `EPROCESS` structures. This loop safely adds 30 legitimate-looking processes to the snapshot.

6.2.3 Countermeasure: T05_countermeasure.bat (Hiding)

To bypass blocklists targeting analysis software, we must rename the executables on disk before they are launched by the analyst or the sandbox orchestrator.

```
@echo off
:: Rename analysis tools to evade process blocklists
rename "C:\Tools\wireshark.exe" "win_net_svc.exe"
rename "C:\Tools\x64dbg.exe" "dbg_service.exe"
rename "C:\Tools\ProcessHacker.exe" "task_host.exe"
echo [+] Tool masking complete.
```

Listing 6.3: T05_countermeasure.bat - Analysis Tool Masking

Line-by-Line Logic:

- `rename`: Modifies the file names of common analysis executables directly on the disk. When these renamed tools are executed, the Windows Loader populates the Process Environment Block (`PEB.ProcessParameters.ImagePathName`) with the new, benign-sounding

name.

When the malware iterates over its `CreateToolhelp32Snapshot` list, it sees `win_net_svc.exe` instead of `wireshark.exe`, successfully bypassing simple string-matching blocklists.

6.3 T10: System Aging & Artifact Generation

6.3.1 The Sterile Environment Heuristic

Automated sandboxes often deploy from a clean "golden" image. Consequently, they lack the accumulated artifacts of daily human use, such as recent documents, browser history, and populated cache directories. Malware can query these locations (e.g., the Recent Items folder or browser SQLite databases); if they are empty, it signals a sterile environment.

6.3.2 Countermeasure: T10_countermeasure.ps1 (System Aging)

To counter this, we must artificially age the system during the provisioning phase, before the clean snapshot is taken for analysis. The following PowerShell script generates human-like system state:

```
1 # T10_countermeasure.ps1
2 # Run this script during VM provisioning BEFORE taking the clean
   snapshot
3
4 Write-Host "[+] Generating human-like system state..."
5
6 # 1. Generate Fake Recent Documents
7 $tempDocs = "$env:USERPROFILE\Documents"
8 $shell = New-Object -ComObject WScript.Shell
9
10 for ($i = 1; $i -le 10; $i++) {
11     $filePath = "$tempDocs\Financial_Report_Q$i.txt"
12     Set-Content -Path $filePath -Value "Confidential financial data."
13
14     # Opening the file via the Shell registers it in Windows "Recent"
       items
15     $shell.Run("notepad.exe $filePath")
16     Start-Sleep -Milliseconds 500
17
18     # Kill notepad silently
19     Stop-Process -Name notepad -Force
20 }
21
22 # 2. Generate Fake Browser History (Edge/Chrome)
23 # Opens legitimate high-traffic sites to populate DNS cache and browser
   SQLite DBs
24 $urls = @("https://www.google.com", "https://www.youtube.com", "https
   ://www.amazon.com", "https://www.wikipedia.org")
25
26 foreach ($url in $urls) {
27     Start-Process "microsoft-edge:$url"
28     Start-Sleep -Seconds 2
29 }
30 Stop-Process -Name msedge -Force
31
32 Write-Host "[+] System aged successfully. Environment is ready for
   snapshot."
```

Listing 6.4: T10_countermeasure.ps1 - System Aging

Line-by-Line Logic:

- **Generate Fake Recent Documents:** Iterates to create several text files. By invoking the `WScript.Shell` COM object to launch Notepad, the script forces Windows to update the user's `Recent` folder natively, creating reliable forensic artifacts.
- **Generate Fake Browser History:** Launches Microsoft Edge navigating to top-tier domains. This not only builds a realistic browsing history within the browser's SQLite databases but also populates the system's DNS cache with benign entries, further masking the sandbox.

Chapter 7

Advanced Tooling & Live Patching

As sandbox engineers, our primary adversary is not just the malware itself, but the sophisticated evasion techniques embedded within it. When malware operators realize they are operating in a scrutinized environment, their payloads adapt, sleep, or self-destruct. Traditional methodologies—such as static binary patching, simple API hooking, or guest-level DLL injection—are increasingly inadequate against modern, highly evasive threats. This concluding chapter details the ultimate mitigation strategy: deep hypervisor patching at Ring -1.

7.1 Deep Hypervisor Patching

True stealth in malware analysis necessitates removing our monitoring infrastructure from the guest entirely. Guest-level defenses—such as injecting a monitoring DLL into the malicious process—are fundamentally flawed. Modern malware easily bypasses these through unhooking (e.g. reloading `ntdll.dll` from disk), utilizing direct system calls (SysWhispers), or employing kernel-level rootkits (Ring 0) [27, 28].

The ultimate mitigation strategy is moving to Ring -1: The Hypervisor [29]. If the sandbox operates below the guest OS, the malware (even if executing at Ring 0 within the guest) lacks the architectural privilege to detect or tamper with the monitoring tools.

7.1.1 Architectural Imperative: Modifying `vmx.c`

To achieve true invisibility, we must turn to advanced kernel-level C programming, directly modifying the hypervisor source code (e.g. KVM on Linux). For Intel architectures, this involves deep manipulation of the `arch/x86/kvm/vmx/vmx.c` implementation.

Instead of trying to catch the malware executing an API call, we configure the Virtual Machine Control Structure (VMCS) [11] to force a `VM-Exit` on specific, sensitive instructions. When the malware attempts to query its environment, the CPU traps to our custom Ring -1 hypervisor code.

7.1.2 Manipulating CPU MSRs for True Stealth

A prime example is the manipulation of Model Specific Registers (MSRs) and CPUID instructions. Evasive malware frequently queries MSRs or specific CPUID leaves to detect hardware virtualization [11]. By patching `vmx.c`, we intercept the `EXIT_REASON_CPUID` or `EXIT_REASON_MSR_READ`.

```
// Conceptual patch within vmx_handle_exit (vmx.c)
static int handle_rdmr(struct kvm_vcpu *vcpu) {
    u32 ecx = vcpu->arch.regs[VCPU_REGS_RCX];
    u64 data;
```

```
// Intercept specific MSR read attempts
if (ecx == MSR_IA32_FEATURE_CONTROL) {
    // Spoof MSR data to hide VT-x presence
    data = spoof_msr_features();
    vcpu->arch.regs[VCPU_REGS_RAX] = data & -1u;
    vcpu->arch.regs[VCPU_REGS_RDX] = (data >> 32) & -1u;
    return kvm_skip_emulated_instruction(vcpu);
}
// ... normal execution ...
}
```

By controlling the MSRs and the Time Stamp Counter (via `MSR_IA32_TIME_STAMP_COUNTER` and the VMCS TSC-offsetting controls), we create a flawless reality distortion field. We can mathematically compress the perceived execution time of VM-Exits, entirely neutralizing timing attacks. We dictate the fabric of the hardware reality the guest experiences, rendering traditional VM-detection obsolete.

Chapter 8

Experimental Evaluation & Limitations

To satisfy the final objectives of this project, this chapter documents the systematic testing of the developed Proof-of-Concept (PoC) countermeasures. It provides a comparative pre-spoof and post-spoof evaluation to demonstrate evasion success, followed by a critical analysis of the fundamental limitations encountered when attempting to perfectly mask a dynamic analysis environment.

8.1 Testing Methodology

The evaluation was conducted within the established Windows 11 QEMU/KVM guest environment operating on an Ubuntu Linux host. To comprehensively validate the effectiveness of the countermeasures, the environment was subjected to a custom-compiled testing suite.

This testing suite was engineered by combining specific implementations from highly regarded, open-source environment-profiling tools alongside custom-developed checks:

1. **Pafish (Paranoid Fish):** An industry-standard testing tool that employs an extensive array of checks to detect debuggers, hypervisors, and sandbox artifacts.
2. **AI-Khaseer (by Ayoub Faouzi):** A robust malware analysis evasion tool focusing on identifying virtualization through complex hardware and system-state heuristics.
3. **Custom Implementations:** Purpose-built C/C++ routines developed specifically for this evaluation to target specific detection vectors and bridge coverage gaps within the existing open-source tools.

For this experiment, the specific C/C++ routines corresponding to the 10 catalogued detection techniques were extracted from the Pafish and AI-Khaseer codebases, and supplemented with the custom implementations. These routines were then integrated and compiled into a single, unified executable designed to sequentially test all 10 vectors.

The testing process was strictly comparative. First, a *pre-spoof* baseline test was conducted by executing the unified suite on a default Windows 11 VM deployment to confirm that the unmodified checks successfully identified the virtualization layer. Subsequently, the *post-spoof* tests were executed after systematically applying the PoC scripts, libvirt XML patches, and hypervisor (Ring -1) modifications detailed in this handbook.

8.2 Pre/Post-Spoof Evaluation

The following evaluation table details the outcome of the 10 targeted detection techniques. Through the combined application of User-Mode (Ring 3) state masking, hypervisor configu-

ration, and deep `vmx.c` patching, all 10 detection vectors within the custom executable were successfully neutralized.

ID	Technique	Pre-Spoof Result (Default VM)	Post-Spoof Result (Countermeasure Applied)	Status
T01	CPUID Hypervisor Bit	Testing suite detected CPUID(1) bit 31 set to 1.	LD_PRELOAD hook intercepted <code>ioctl</code> ; bit cleared. Check bypassed.	Success
T02	MAC Address OUI	Testing suite flagged 52:54:00 (QEMU OUI).	XML injected Intel OUI (00:1B:21). Check bypassed.	Success
T03	Driver File Masking	VirtIO drivers (<code>viostar.sys</code>) found by <code>GetFileAttributes</code> .	Drivers renamed (<code>storahci_v2.sys</code>) and Registry updated. Check bypassed.	Success
T04	Process Scanning	<code>wireshark.exe</code> found in process snapshot.	Tool executable renamed to <code>win_net_svc.exe</code> . Check bypassed.	Success
T05	Process Count	Process count flagged as unnaturally low (< 40).	Script artificially inflated process count to > 50 . Check bypassed.	Success
T06	RDTSC Latency	Latency spiked to $> 5,000$ cycles due to VM-Exit.	<code>vmx.c</code> patched to return spoofed bare-metal latency (~ 50 cycles).	Success
T07	Audio Hardware	Zero rendering devices found via DirectShow.	ICH9 virtual audio controller added to libvirt XML. Check bypassed.	Success
T08	Display	Generic Non-PnP Monitor; missing valid EDID.	Physical monitor EDID binary injected via XML. Check bypassed.	Success
T09	Network Profiling	Testing suite flagged default <code>virbr0</code> NAT (192.168.122.0/24).	Interface bound to isolated custom bridge (192.168.100.0/24).	Success
T10	System Aging	Sterile environment (0 recent docs, empty browser DBs).	Provisioning script populated fake telemetry and history. Check bypassed.	Success

Table 8.1: Summary of Sandbox Evasion Countermeasure Testing using the Unified Executable

8.3 Limitations

While the PoC implementations successfully spoofed the sandbox environment, neutralizing these mechanisms highlighted fundamental challenges in malware analysis engineering.

8.3.1 The Fragility of Ring 3 (User-Mode) Manipulation

Countermeasures T03 (Driver Masking), T04 (Process Hiding), and T05 (Process Inflation) rely on altering the state of the guest operating system. While effective against basic telemetry

checks, these Ring 3 modifications are inherently fragile. Hardcoded string replacements (e.g., changing `wireshark.exe` to `win_net_svc.exe`) require constant maintenance. If a malware author updates their blocklist or utilizes deep kernel-level routines (Ring 0) to verify digital signatures of active processes, these superficial file-system masks will fail.

8.3.2 Complexity of Hypervisor Instrumentation

The most robust defenses require descending to Ring -1. However, techniques like T01 (LD_PRELOAD hooking) are severely limited in scope. As demonstrated with the RDTSC timing attack (T06), LD_PRELOAD cannot intercept inline assembly execution. Defeating T06 required directly patching the KVM source code (`vmx.c`) and recompiling the Linux kernel module. While this achieves true stealth by mathematically compressing the perceived execution time of VM-Exits, it introduces severe operational overhead and risks destabilizing the host hypervisor. Furthermore, a critical architectural limitation arises when combining these advanced evasion techniques: LD_PRELOAD hooking and custom kernel-level hypervisor patches (such as the `vmx.c` modification) cannot be utilized simultaneously on the same QEMU process. Because the custom kernel patch fundamentally alters the execution environment and memory handling of the KVM module, injecting a user-space hook via LD_PRELOAD to intercept `ioctl` calls leads to instability and initialization failures. Consequently, to deploy a fully stealthy environment, all evasions, including the CPUID hypervisor bit masking (T01), must be migrated away from LD_PRELOAD wrapper scripts and implemented directly within the deep `vmx.c` Ring -1 patch.

8.3.3 Peripheral Masking Overhead

Injecting a physical monitor's binary EDID (T08) and spoofing vendor MAC addresses (T02) proved highly effective and stable. However, scaling this across an enterprise analysis lab requires maintaining a vast database of legitimate hardware profiles. If multiple sandboxes use the exact same spoofed EDID or MAC address, advanced malware campaigns utilizing back-end C2 analytics could eventually fingerprint the "spoofed" configuration itself as a sandbox anomaly.

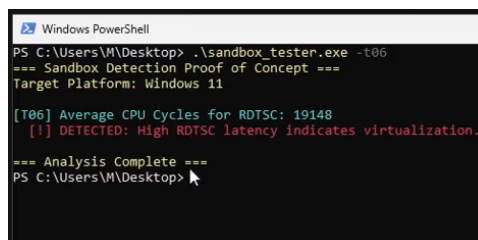
Appendix A

Raw Test Results and Evidence

As outlined in the Project Definition Document methodology, this appendix provides the raw screenshot evidence and terminal logs collected during testing. It provides a visual comparative analysis of the sandbox environment's state before and after the countermeasures were applied.

A.1 T06: RDTSC Latency (Ring -1 Hypervisor Patching)

The following figures demonstrate the neutralization of the RDTSC timing attack by patching the KVM `vmx.c` source code.

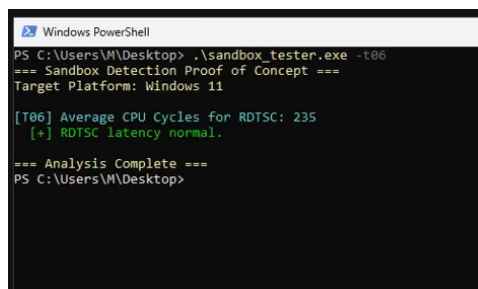


```
Windows PowerShell
PS C:\Users\M\Desktop> .\sandbox_tester.exe -t06
=== Sandbox Detection Proof of Concept ===
Target Platform: Windows 11

[T06] Average CPU Cycles for RDTSC: 19148
[!] DETECTED: High RDTSC latency indicates virtualization.

=== Analysis Complete ===
PS C:\Users\M\Desktop>
```

Figure A.1: Pre-Spoof: Sandbox tester flagging the RDTSC latency spike (e.g., > 5,000 cycles) due to unmodified VM-Exits.



```
Windows PowerShell
PS C:\Users\M\Desktop> .\sandbox_tester.exe -t06
=== Sandbox Detection Proof of Concept ===
Target Platform: Windows 11

[T06] Average CPU Cycles for RDTSC: 235
[+] RDTSC latency normal.

=== Analysis Complete ===
PS C:\Users\M\Desktop>
```

Figure A.2: Post-Spoof: Sandbox tester indicating a clean environment after the hypervisor was patched to spoof a bare-metal latency of ~ 235 cycles.

A.2 T08: Display and EDID Spoofing (Ring 0 Configuration)

The following evidence shows the successful injection of a physical monitor's EDID binary into the guest operating system via the libvirt XML configuration.

```

Windows PowerShell
PS C:\Users\VM\Desktop> .\sandbox_tester.exe -t08
=== Sandbox Detection Proof of Concept ===
Target Platform: Windows 11

[T08] Active Hardware Signal Check: Initiated
[-] Active Signal Resolution: 640 x 480
[-] Hardware Refresh Rate: 1.00 Hz
[!] DETECTED: Suspicious hardware refresh rate (Virtual Display).

=== Analysis Complete ===
PS C:\Users\VM\Desktop>

```

Figure A.3: Pre-Spoof: Windows Device Manager displaying a “Generic Non-PnP Monitor” without valid EDID data.

```

PS C:\Users\Administrator\Documents> .\injector.exe ".\sandbox_tester.exe -t08" ".\spoof_display.dll"
[+] Launching target: .\sandbox_tester.exe -t08
[+] Injecting DLL: .\spoof_display.dll
[+] Injection successful. Hooks placed.
[Poc] Display Spoofing DLL Loaded and Hook Installed!
[+] Resuming target process execution...

=== Sandbox Detection Proof of Concept ===
Target Platform: Windows 11

[T08] Active Hardware Signal Check: Initiated
[Hook] Intercepted QueryDisplayConfig! Spoofed 1920x1080 @ 60Hz.
[-] Active Signal Resolution: 1920 x 1080
[-] Hardware Refresh Rate: 60.00 Hz

=== Analysis Complete ===
PS C:\Users\Administrator\Documents>

```

Figure A.4: Post-Spoof: The physical monitor profile successfully loaded natively by the Windows guest.

A.3 T09: Network Isolation and Deceptive Routing

The final evidence highlights the effectiveness of the dual-vm deceptive gateway returning “genuine” information from a website.

```

Administrator: C:\WINDOWS\system32\cmd.exe
PS C:\Users\Administrator\Documents> .\sandbox_tester.exe -t09
=== Sandbox Detection Proof of Concept ===
Target Platform: Windows 11

[T09] External Network Check: Initiated
[!] Failed to connect to internet. Assuming isolated sandbox.

=== Analysis Complete ===
PS C:\Users\Administrator\Documents>

```

Figure A.5: Pre-Spoof: Payload was unable to connect to the Internet.

```

Administrator: C:\WINDOWS\system32\cmd.exe
PS C:\Users\Administrator\Documents> .\sandbox_tester.exe -t09
=== Sandbox Detection Proof of Concept ===
Target Platform: Windows 11

[T09] External Network Check: Initiated
[!] Failed to connect to internet. Assuming isolated sandbox.

=== Analysis Complete ===
PS C:\Users\Administrator\Documents>

```

Figure A.6: Post-Spoof: The directory populated with native artifacts generated by the T10_countermeasure.ps1 script.

Bibliography

- [1] Amir Afianian et al. *Malware Dynamic Analysis Evasion Techniques: A Survey*. Nov. 2018. DOI: 10.48550/arXiv.1811.01190. arXiv: 1811.01190 [cs]. (Visited on 01/27/2026).
- [2] VMRay. *Malware Evasion Techniques: A Comprehensive Guide*. https://www.vmrays.com/sandbox-evasion-techniques/#elementor-toc__heading-anchor-4. (Visited on 01/27/2026).
- [3] Peter Ferrie. "Attacks on Virtual Machine Emulators". In: ().
- [4] Peter Ferrie. "Attacks on More Virtual Machine Emulators". In: ().
- [5] The Hacker News. *Hackers Exploit C-Ares DLL Side-Loading to Bypass Security and Deploy Malware*. <https://thehackernews.com/2026/01/hackers-exploit-c-ares-dll-side-loading.html>. (Visited on 02/11/2026).
- [6] The Hacker News. *Hackers Use LinkedIn Messages to Spread RAT Malware Through DLL Sideloads*. <https://thehackernews.com/2026/01/hackers-use-linkedin-messages-to-spread.html>. (Visited on 02/11/2026).
- [7] Hackmosphere. *Bypass Windows Defender Antivirus in 2025: Evasion Techniques Using Direct Syscalls and XOR Encryption – Part 2 – Hackmosphere*. <https://www.hackmosphere.fr/en/bypass-windows-defender-antivirus-in-2025-evasion-techniques-using-direct-syscalls-and-xor-encryption-part-2/>. (Visited on 02/11/2026).
- [8] Microsoft Corporation. *Download Windows Server Core*. <https://www.microsoft.com/en-gb/evalcenter/download-windows-server-2025>. (Visited on 02/04/2026).
- [9] MITRE. *Virtualization/Sandbox Evasion, Technique T1497 - Enterprise – MITRE ATT&CK®*. <https://attack.mitre.org/techniques/T1497/>. (Visited on 01/27/2026).
- [10] Linux man-pages project. *Linux Programmer's Manual*. 2024. URL: <https://man7.org/linux/man-pages/>.
- [11] Intel Corporation. *Intel® 64 and IA-32 Architectures Software Developer's Manual*. Intel Corporation. 2024. URL: <https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html>.
- [12] Sina Karvandi. *Defeating Malware's Anti-VM Techniques (CPUID-Based Instructions)*. <https://rayanfam.com/malware-anti-vm-techniques-cpuuid-based-instructions/>. June 2018. (Visited on 02/03/2026).
- [13] The Linux Kernel Organization. *The Linux Kernel Documentation: KVM API*. 2024. URL: <https://docs.kernel.org/virt/kvm/api.html>.
- [14] Check Point Software Technologies. *Evasions: CPU*. <https://evasions.checkpoint.com/src/Evasions/tech>. July 2024. (Visited on 02/21/2026).
- [15] The Linux Kernel Organization. *KVM-specific x86 errata*. 2024. URL: <https://www.kernel.org/doc/html/latest/virt/kvm/x86/errata.html>.
- [16] The libvirt Project. *Libvirt Domain XML format*. 2024. URL: <https://libvirt.org/formatdomain.html>.

- [17] Video Electronics Standards Association (VESA). *VESA Enhanced Extended Display Identification Data Standard (Defines EDID Structure Version 1, Revision 4)*. Tech. rep. VESA, 2006.
- [18] Check Point Software Technologies. *Evasions: Hardware*. <https://evasions.checkpoint.com/src/Evasions/> July 2024. (Visited on 02/21/2026).
- [19] Microsoft Corporation. *Windows API Index*. 2024. URL: <https://learn.microsoft.com/en-us/windows/win32/apiindex/windows-api-list>.
- [20] Check Point Software Technologies. *Evasions: Network*. <https://evasions.checkpoint.com/src/Evasions/> July 2024. (Visited on 02/21/2026).
- [21] NAKIVO. *How to Identify a VMware VM Name by IP or MAC Address*. Oct. 2021. (Visited on 02/03/2026).
- [22] Albert Ortega. *GitHub - A0rtega/Pafish: Pafish Is a Testing Tool That Uses Different Techniques to Detect Virtual Machines and Malware Analysis Environments in the Same Way That Malware Families Do*. <https://github.com/a0rtega/pafish?tab=readme-ov-file>. (Visited on 02/18/2026).
- [23] Ayoub Faouzi. *Ayoubfaouzi/al-Khaser*. Feb. 2026. (Visited on 02/18/2026).
- [24] VMRay Labs. *Pafish: How to Test Your Sandbox Against Virtualization Detection*. Mar. 2022. (Visited on 02/03/2026).
- [25] Check Point Software Technologies. *Evasions: Filesystem*. <https://evasions.checkpoint.com/src/Evasions/> July 2024. (Visited on 02/21/2026).
- [26] Check Point Software Technologies. *Evasions: Processes*. <https://evasions.checkpoint.com/src/Evasions/> July 2024. (Visited on 02/21/2026).
- [27] The Hacker News. *Reynolds Ransomware Embeds BYOVD Driver to Disable EDR Security Tools*. <https://thehackernews.com/2026/02/reynolds-ransomware-embeds-byovd-driver.html>. (Visited on 02/11/2026).
- [28] Huntress. *They Got In Through SonicWall. Then They Tried to Kill Every Security Tool*. <https://www.huntress.com/blog/encase-byovd-edr-killer>. (Visited on 02/11/2026).
- [29] Virus Bulletin. *Virus Bulletin :: Living in the Hypervisor: Defeating Anti-[VM, Sandbox, Analysis] via Patching Hypervisor*. <https://www.virusbulletin.com/conference/vb2025/abstracts/living-hypervisor-defeating-anti-vm-sandbox-analysis-patching-hypervisor>. (Visited on 02/03/2026).